

实验六 ALOHA

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 12 月 21 日

一、实验背景

实验目的：

加深对纯 ALOHA 和 Slotted-ALOHA 分析的理解并通过仿真观察随机接入现象、验证理论结果。

Matlab 版本：2023b。

二、实验内容

1. 实验步骤

- (1) 设置用户数 N 和单用户的帧到达率 λ ，分别绘制出纯 ALOHA 和 Slotted-ALOHA 的理论计算的 ρ - G 曲线和在上述两种帧到达过程设置下的仿真 ρ - G 曲线（设置 2 下先不修正）。观察结果，并解释仿真结果和理论结果有差距的原因。
- (2) 在(1)的绘图结果的基础上多绘制一条仿真 ρ - G 曲线：在帧到达过程设置 2 下修正使之更接近于真实的到达率，观察结果。代码中给出了一种修正方法，解释它的原理。
- (3) 改变用户数 N ，重做(2)，观察总结 N 对仿真结果的影响，并解释。

2. 理论推导

纯 ALOHA：归一化总业务量 $\rho = Ge^{-2G}$ ，易得在 $G=0.5$ 时， ρ 取得最大值 $\frac{1}{2e}$

Slotted-ALOHA：归一化总业务量 $\rho = Ge^{-G}$ ，易得在 $G=1$ 时， ρ 取得最大值 $\frac{1}{e}$

3. 代码实现

纯 ALOHA：

```
%% 纯 ALOHA 仿真
clc
clear

% 设置系统参数
N = 100;% 用户数
lambdas = 0.1:0.5:50;% 单个用户的帧到达率集合
b = 100;
R = 1e5;
T_fr = b/R;
T = 1e2;% 总仿真随机接入时长，可以根据仿真情况适当修改
```

```
lambdalength = length(lambdas);
rho_simu1 = zeros(size(lambdas));% 根据纯指数分布生成所有用户的帧到达情况下的总归一化吞吐速率
rho_simu2 = zeros(size(lambdas));% 设置某一用户在发送帧的过程中不会有新的帧到达该用户时计算的总归一化吞吐速率
tic
for lambdaCounter = 1:lambdalength

    lambda = lambdas(lambdaCounter);
    MAX_number_of_arrived_frames_in_T_for_each_user=ceil(T*lambda + 5*sqrt(T*lambda));

    % 生成所有用户的帧到达情况-到达时刻与所属用户
    TotalFrameArrivalConditions1 = [];% 根据纯指数分布生成所有用户的帧到达情况
    TotalFrameArrivalConditions2 = [];% 在指数分布的情况下，设置某一用户在发送帧的过程中不会有新的帧到达该用户
    for n = 1:N

        % 生成 TotalFrameArrivalConditions1---

        FrameArrivalTimes=zeros(1,MAX_number_of_arrived_frames_in_T_for_each_user); % 预开一个数组用来存储当前用户的 T 时间内的所有帧到达时刻，同学们要注意避免溢出

        % 采用逐个累加的方式生成 T 时间内的所有帧到达时刻
        number_of_arrived_frames = 0;
        FrameArrivalTime = exprnd(1/lambda);% 第一帧的到达时刻

        while FrameArrivalTime<T
            % 按照指数分布生成帧到达时刻
            number_of_arrived_frames = number_of_arrived_frames+1;
            FrameArrivalTimes(number_of_arrived_frames) =
FrameArrivalTime;
            FrameArrivalTime = FrameArrivalTime+exprnd(1/lambda);% 当前最新一帧的到达时刻
        end
        CurrentUserFrameArrivals =
[FrameArrivalTimes(1:number_of_arrived_frames);ones(1,number_of_arrived_frames)*n];
        TotalFrameArrivalConditions1 =
[TotalFrameArrivalConditions1,CurrentUserFrameArrivals];
```

```
% 生成 TotalFrameArrivalConditions2---
FrameArrivalTimes=zeros(1,MAX_number_of_arrived_frames_in_T_for_
each_user);
number_of_arrived_frames = 0;
FrameArrivalTime = exprnd(1/lambda);
while FrameArrivalTime<T
    number_of_arrived_frames = number_of_arrived_frames+1;
    FrameArrivalTimes(number_of_arrived_frames) =
FrameArrivalTime;
    FrameArrivalTime = FrameArrivalTime+T_fr+exprnd(1/lambda);
end
CurrentUserFrameArrivals =
[FrameArrivalTimes(1:number_of_arrived_frames);ones(1,number_of_arrived_
_frames)*n];
TotalFrameArrivalConditions2 =
[TotalFrameArrivalConditions2,CurrentUserFrameArrivals];
end

% 统计仿真时长 T 内总的成功发送的帧的数量并计算归一化吞吐速率
% 先所有用户的帧到达时刻按从先到后排序
[~,SortedIdxs1] = sort(TotalFrameArrivalConditions1(1,:));
TotalFrameArrivalConditions1 =
TotalFrameArrivalConditions1(:,SortedIdxs1);
[~,SortedIdxs2] = sort(TotalFrameArrivalConditions2(1,:));
TotalFrameArrivalConditions2 =
TotalFrameArrivalConditions2(:,SortedIdxs2);

% 送入函数 SuccessFrameNumCalcu 统计成功传输帧的数量并计算归一化吞吐
速率 (这里计算的是所有用户总的归一化吞吐速率, 同学们也可观察每一个用户的情况)
rho_simu1(lambdaCounter) =
sum(SuccessFrameNumCalcu(TotalFrameArrivalConditions1,T_fr,N))/T*T_fr;
rho_simu2(lambdaCounter) =
sum(SuccessFrameNumCalcu(TotalFrameArrivalConditions2,T_fr,N))/T*T_fr;
end
toc

figure
G_s = 0:0.01:N*lambda_s(end)*T_fr;
plot(G_s,G_s.*exp(-2*G_s),'DisplayName','纯 Aloha 理论值')
hold on
plot(N*lambda_s*T_fr,rho_simu1,'+', 'DisplayName','纯 Aloha 仿真值')
hold on
plot(N*lambda_s*T_fr,rho_simu2,'*', 'DisplayName','纯 Aloha 仿真值-用户自己
帧碰撞避免(到达率校正前)')
```

```
hold on
plot(N*(lambdas./(1+lambdas*T_fr))*T_fr,rho_simu2,'hexagram','DisplayNa
me','纯 Aloha 仿真值-用户自己帧碰撞避免(到达率校正后)')
legend
grid on
title(['N = ', num2str(N)])
xlabel('G')
ylabel('\rho')

function SuccessFrameNum_Users =
SuccessFrameNumCalcu(TotalFrameArrivalConditions,T_fr,N)
    % 输入: TotalFrameArrivalConditions: 矩阵, 两行, 第一行-所有用户的帧到
    达时刻(按照从先到后顺序),
    % 第二行-帧所属用户编号; T_fr: 标量, 无碰撞时一帧的发送耗时; N: 标量, 用
    户总数
    % 输出: SuccessFrameNum_Users: 矢量, 每个用户成功传输(不与别的帧碰撞)的
    帧的数量
    % 函数功能: 在帧到达即发送的情况下, 统计每个用户能成功传输的帧的数量

    TotalFrameArrivalTimes = TotalFrameArrivalConditions(1,:);
    TotalFrameOwners = TotalFrameArrivalConditions(2,:); % 修正此处的索引
    TotalFrameNum = length(TotalFrameArrivalTimes);
    TransmissionConditions = zeros(1, TotalFrameNum);

    for i = 1:TotalFrameNum
        % 检查前一帧: 如果是第一帧, 无前一帧冲突
        pre_ok = (i == 1) || (TotalFrameArrivalTimes(i) >=
TotalFrameArrivalTimes(i-1) + T_fr);
        % 检查后一帧: 如果是最后一帧, 无后一帧冲突
        next_ok = (i == TotalFrameNum) ||
(TotalFrameArrivalTimes(i+1) >= TotalFrameArrivalTimes(i) + T_fr);
        if pre_ok && next_ok
            TransmissionConditions(i) = 1;
        end
    end

    SuccessFrameNum_Users = zeros(1, N);
    for n = 1:N
        SuccessFrameNum_Users(n) =
sum(TotalFrameOwners(logical(TransmissionConditions)) == n);
    end
end
```

Slotted-ALOHA:

```
%% Slotted ALOHA 仿真
clc
clear
% 设置系统参数
N = 100;% 用户数
lambdas = 0.1:0.5:50;% 单个用户的帧到达率集合
b = 100;
R = 1e5;
T_fr = b/R;
T = 1e2;%总仿真随机接入时长，可以根据仿真情况适当修改

lambdalength = length(lambdas);
rho_simu1 = zeros(size(lambdas));
rho_simu2 = zeros(size(lambdas));%设置某一用户在发送帧的过程中不会有新的帧
到达该用户时计算的归一化吞吐速率

tic

for lambdaCounter = 1:lambdalength
    lambda = lambdas(lambdaCounter);
    % 生成所有用户的帧到达时刻-TotalFrameArrivalTimes
    MAX_number_of_arrived_frames_in_T_for_each_user=ceil(T*lambda +
5*sqrt(T*lambda));
    slots1=zeros(1,ceil(T/T_fr));
    slots2=zeros(1,ceil(T/T_fr));
    %N 个用户
    for n = 1:N
        FrameArrivalTimes=zeros(1,MAX_number_of_arrived_frames_in_T_for_
each_user);
        number_of_arrived_frames=0;
        FrameArrivalTime = exprnd(1/lambda);
        while FrameArrivalTime<T
            %按照指数分布生成帧到达时刻
            number_of_arrived_frames=number_of_arrived_frames+1;
            FrameArrivalTimes(number_of_arrived_frames) =
FrameArrivalTime;
            FrameArrivalTime = FrameArrivalTime+exprnd(1/lambda);
        end
        %将每个包在对应时间的 slot 发送，并记录该 slot 的使用情况
        for i=1:number_of_arrived_frames
            slot_index=ceil(FrameArrivalTimes(i)/T_fr);
            slots1(slot_index)=slots1(slot_index)+1;
        end
    end
end
```

```
        FrameArrivalTimes=zeros(1,MAX_number_of_arrived_frames_in_T_for_
each_user);
        number_of_arrived_frames=0;
        FrameArrivalTime = exprnd(1/lambda);
        while FrameArrivalTime<T
            %设置一个用户不会和自己的帧产生冲突
            number_of_arrived_frames=number_of_arrived_frames+1;
            FrameArrivalTimes(number_of_arrived_frames) =
FrameArrivalTime;
            FrameArrivalTime = FrameArrivalTime+T_fr+exprnd(1/lambda);
        end
        for i=1:number_of_arrived_frames
            slot_index=ceil(FrameArrivalTimes(i)/T_fr);
            slots2(slot_index)=slots2(slot_index)+1;
        end
    end
end

% 统计仿真时长 T 内总的成功发送的帧的数量并计算归一化吞吐速率
% 归一化吞吐量 rho = (成功帧数 * 帧长) / (仿真时长 * 发送速率)
rho_simu1(lambdaCounter) = sum(slots1 == 1) * T_fr / T;
rho_simu2(lambdaCounter) = sum(slots2 == 1) * T_fr / T;
end

toc

figure
G_s = 0:0.01:N*lambda_s(end)*T_fr;
plot(G_s,G_s.*exp(-G_s),'DisplayName','Slotted ALOHA 理论值')
hold on
plot(N*lambda_s*T_fr,rho_simu1,'+','DisplayName','Slotted ALOHA 仿真值')
hold on
plot(N*lambda_s*T_fr,rho_simu2,'*','DisplayName','Slotted ALOHA 仿真值(到
达率校正前)')
hold on
plot(N*(lambda_s./(1+lambda_s*T_fr))*T_fr,rho_simu2,'hexagram','DisplayNa
me','Slotted ALOHA 仿真值(到达率校正后)')
legend
grid on
title(['N = ', num2str(N)])
xlabel('G')
ylabel('\rho')
```

3. 实验数据记录及结果分析

- (1) 绘制出纯 ALOHA 和 Slotted-ALOHA 的理论计算的 ρ - G 曲线和在上述两种帧到达过程设置下的仿真 ρ - G 曲线（设置 2 下先不修正），并解释仿真结果和理论结果有差距的原因。

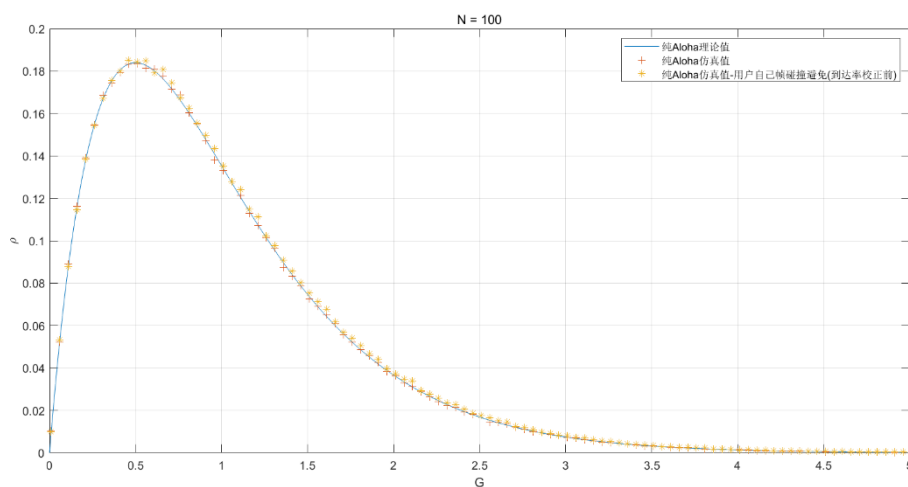


图 1 纯 ALOHA 的 ρ - G 曲线（设置 2 下先不修正）

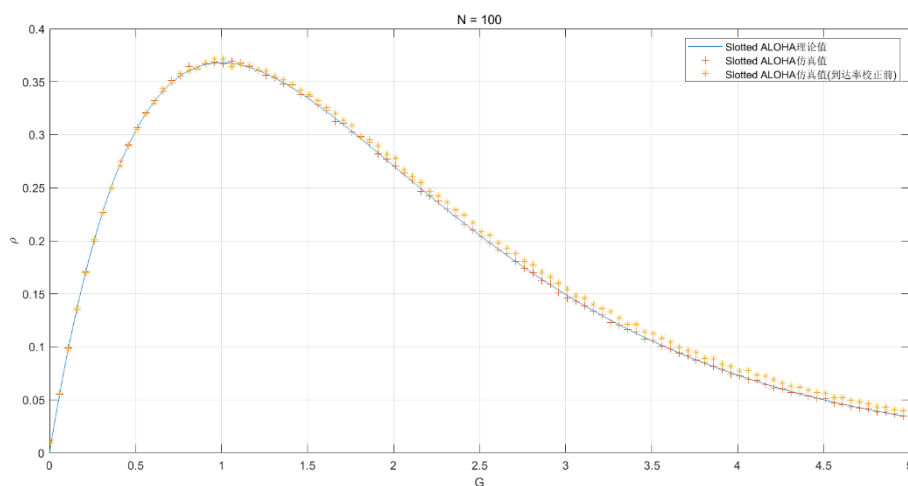


图 2 Slotted-ALOHA 的 ρ - G 曲线（设置 2 下先不修正）

ALOHA 和 Slotted-ALOHA 的 ρ - G 曲线在 2 种实验设置下均基本与理论值相符。具体来看，设置 1（标准泊松过程）的仿真结果与理论值吻合度很高。而设置 2（避免自碰撞且未校正 λ ）的仿真结果在 G 较大时会略高于理论值，这种偏差在 Slotted-ALOHA 中相较 ALOHA 更明显。

其原因是设置 2 使得帧到达不再是完全独立的泊松过程，虽然理论验证了 $N \rightarrow \infty$ 时这种设置与理论值的高度一致，但在 N 有限时会存在一定偏差。而这种偏差在 Slotted-ALOHA 中更明显，是因为 Slotted-ALOHA 要求所有帧的发送时刻必须对齐于离散的时隙起点，也改变

了帧到达的时间结构，进一步降低了帧冲突的随机性。当时隙结构与本已受限的到达间隔叠加时，仿真结果会更明显地偏离理想泊松假设，导致吞吐量在高负载时更明显地高于理论值。

(2) 在(1)的绘图结果的基础上多绘制一条仿真 ρ - G 曲线：在帧到达过程设置 2 下修正使之更接近于真实的到达率，观察结果并解释该修正方法的原理。

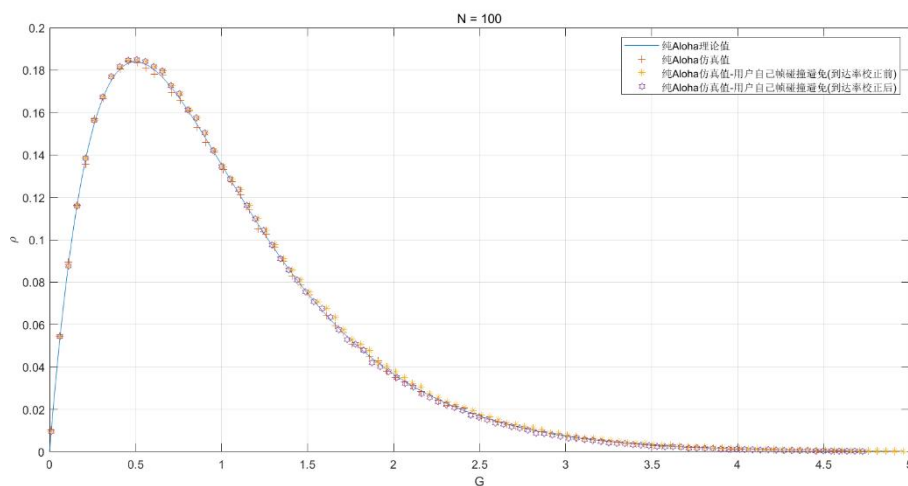


图 3 纯 ALOHA 的 ρ - G 曲线

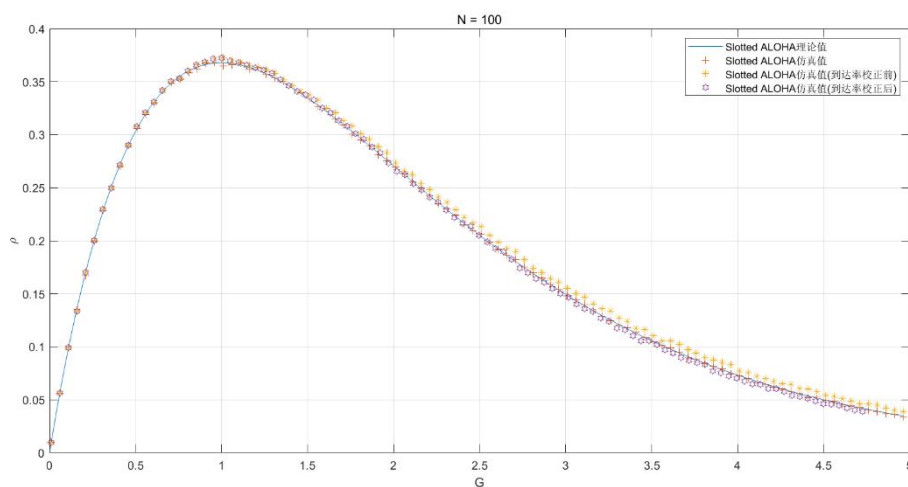


图 4 Slotted-ALOHA 的 ρ - G 曲线

修正原理：原始设置 2 会使得每个用户的帧到达过程不再是泊松过程，此时每个用户的平均周期长度为 $T = T_{fr} + \frac{1}{\lambda}$ ，实际到达率 $\lambda' = \frac{1}{T} = \frac{1}{T_{fr} + \frac{1}{\lambda}} = \frac{\lambda}{\lambda T_{fr} + 1}$ 。

经过修正，ALOHA 和 Slotted-ALOHA 的 ρ - G 曲线在设置 2 下的实验值更接近理论值。具体来说，对应峰值位置的 $G < G_{peak} < \text{对应峰值位置的} G + 1$ 时，仿真值略高于理论值； G 更大时，仿真值略小于理论值。

原因可能是：在中等负载时，消除自碰撞带来的影响仍占主导，使仿真吞吐略高于理论值。而在高负载下，不同用户间的相关性降低了吞吐效率，因此仿真值略低于理论曲线。

(3) 改变用户数 N ，重做(2)，观察总结 N 对仿真结果的影响，并解释。

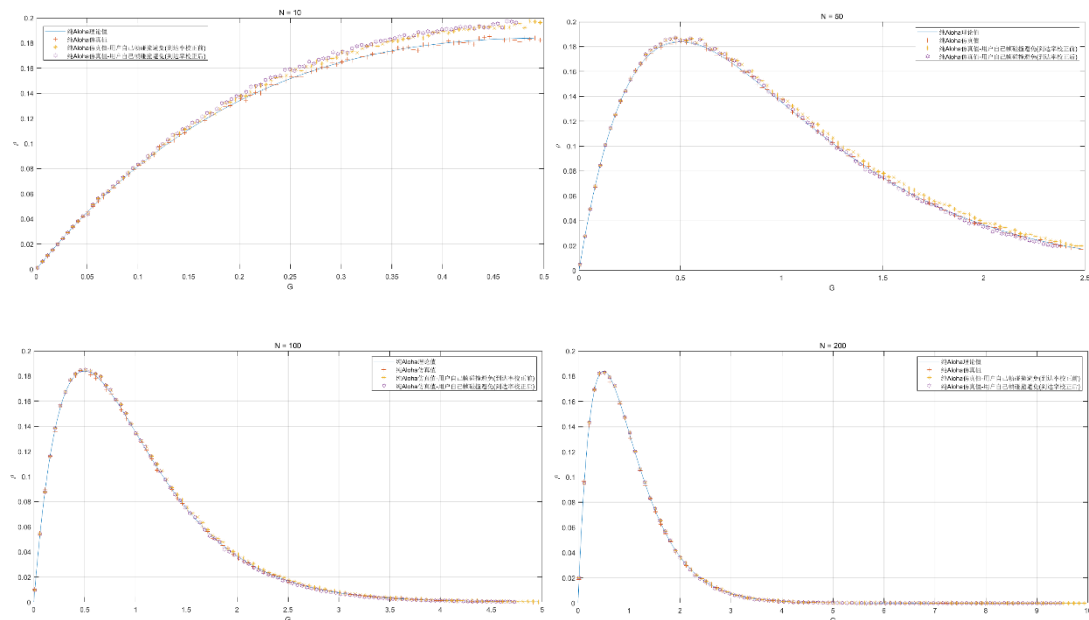


图 5 改变用户数 N 时纯 ALOHA 的 ρ - G 曲线

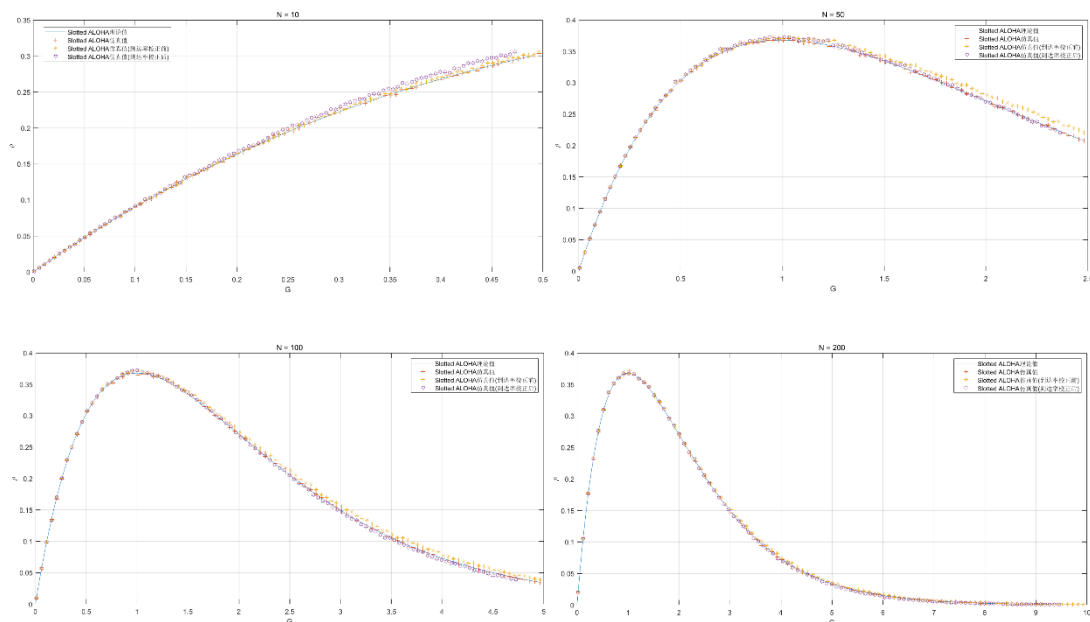


图 6 改变用户数 N 时 Slotted-ALOHA 的 ρ - G 曲线

随着用户数 N 的增加，ALOHA 和 Slotted-ALOHA 的 ρ - G 曲线在 2 种实验设置下的仿真值与理论值吻合度提升，其原因是用户数 N 的增加使实验情况更接近与 $N \rightarrow \infty$ 的理论情况。

三、 实验总结

本次通信与网络课程实验通过 MATLAB 仿真研究了纯 ALOHA 和 Slotted-ALOHA 的随机接入现象。实验首先验证了理论 $\rho-G$ 曲线与设置 1（标准泊松过程）仿真结果的高度吻合，表明理论模型在理想条件下的有效性。在设置 2（避免自碰撞）中，由于帧到达不再是完全独立的泊松过程，因此在 N 有限时会存在一定偏差；经有效到达率的修正后，曲线整体更贴近理论曲线。此外，用户数 N 的增加使仿真结果更接近理论值。

通过本次实验，我基本熟悉了纯 ALOHA 和 Slotted-ALOHA 等核心概念，也提升了 MATLAB 编程的熟练度。